

OPTIONS, PRESSURE, DECISION

The work picked the stack.

Three free roads could ship this portfolio. I chose the one I can maintain and that shows the proof without turning the framework into the story.

3 roads

Genuine alternatives

2 weeks

Finish window tested

4 constraints

Given before options

No backend

Honest launch answer

The decision starts with the work.

AI received the actual audience, content map, display needs, budget, and skill level before it named a tool.

Free only Builder, repository, fonts, hosting, and deployment must work on free tiers.	Honest skill Full-stack developer. React and Git are maintainable; over-engineering is the real risk.
Six-section path Hero → Proof → Work → How I Work → Scope Builder → Contact.	Evidence display Real screenshots, galleries, readable cases, demo/repository links, and occasional embeds.

DYNAMIC AT LAUNCH?

Browser interaction, yes. Server persistence, no.

The scope builder assembles a pre-filled email. There is no account, database, CMS, or stored submission.

Smallest to most powerful.

All three can ship. The trade-off is where complexity appears: repeated markup, a new content framework, or package and deployment maintenance.

01

SMALLEST

Plain HTML
CSS + JS

Host

GitHub Pages

Backend

No

Good: transparent, fast, almost nothing to update.

Cost: repeated case markup and interactive state become manual as the portfolio grows.

02

MIDDLE

Astro
+ Tailwind

Host

Cloudflare / Vercel

Backend

No

Good: content-first components, fast static pages, little client JavaScript.

Cost: migrating the live shell and learning another framework spends the build window.

03

CHOSEN

Vite + React
Tailwind + Motion

Host

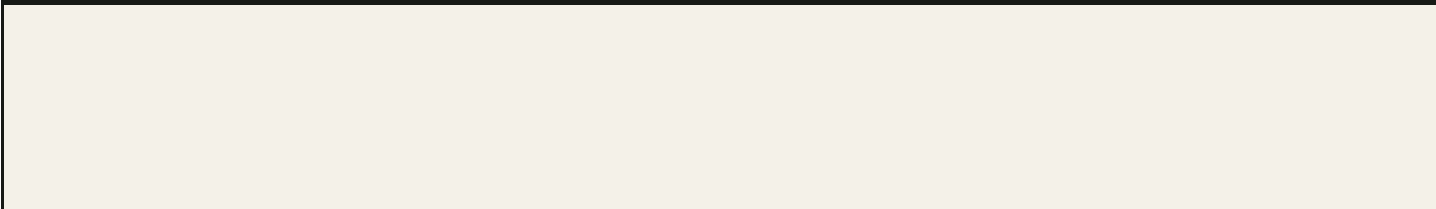
Vercel

Backend

No

Good: repeatable case components, responsive galleries, and scope-builder state.

Cost: dependencies and adapters. The first Vercel build returned 404 until Nitro was configured.



The chosen road gets the hardest questions.

The front-runner must survive actual delivery and display needs, not just sound familiar.

WHAT BREAKS WITH THE SIMPLEST?

Not launch. Change.

Plain HTML can ship everything essential. The drag appears when seven repeated cases and gallery behavior must stay consistent through edits.

WHAT DO I MAINTAIN?

The adapter counts.

Components, case data, Tailwind classes, dependencies, Git, and Vite/vinext/Nitro. The earlier Vercel 404 is the honest maintenance evidence.

CAN I FINISH IN TWO WEEKS?

Yes—with cuts held.

Identity, case copy, images, map, and CTA ladder already exist. No redesign, CMS, account system, or custom backend enters build week.

DOES IT SHOW THE WORK WELL?

Yes, without spectacle.

Responsive galleries, readable expanded cases, explicit demo/repo links, and restrained motion support the proof instead of upstaging it.

React wins—with a maintenance contract.

This is Sara's rationale after the comparison and pressure test.

I chose Vite + React + Tailwind CSS + Framer Motion, hosted on Vercel. Plain HTML would work, but I would maintain repeated project markup as the portfolio grows. Astro is a sensible content road, but migrating now spends the build window on a new framework instead of showing the work.

CAN I MAINTAIN THIS?

Yes. React, Git, component state, and deployment configuration are within my current skill set. Every dependency change still owes me lint, a production build, and a public URL check.

DOES IT SHOW MY WORK WELL?

Yes. Real screenshots, long-form proof, demo/repository links, and the scope builder get one repeatable structure.

BACKEND?

Not yet. Local content plus a browser-built email covers launch. Server code arrives only with a real storage, privacy, or processing need.

Pass / revise

✓ Three genuine options ✓ Free host named ✓ Display needs tested ✓ Maintainability answered ✓ Backend: not yet